

EMBEDDED AI (AD320) – MULTIMODAL EMOTION RECOGNITION SYSTEM

4. Implementation and Results

4.1 System Overview

The developed system is a real-time multimodal emotion recognition framework that combines both textual and facial emotion analysis. The system accepts user text input along with webcam video frames and predicts the final emotional state using confidence-based multimodal fusion.

The overall workflow of the system is as follows:

1. User enters text input through the web interface.
2. Webcam captures the facial image in real time.
3. The text input is processed using a fine-tuned RoBERTa transformer model.
4. The facial image is analyzed using the DeepFace facial emotion recognition model.
5. Confidence scores from both modalities are obtained.
6. A fusion module combines both predictions using weighted confidence logic.
7. The final emotion is displayed on the web interface.

The system supports the following emotion categories:

Angry, Disgust, Fear, Happy, Sad, Surprise, Neutral

4.2 Text Emotion Recognition Module

The text prediction module outputs:

- Predicted emotion label
- Confidence score

Example output:

Input Text:

"I am very happy today!"

Predicted Emotion:
Happy

Confidence:
0.94

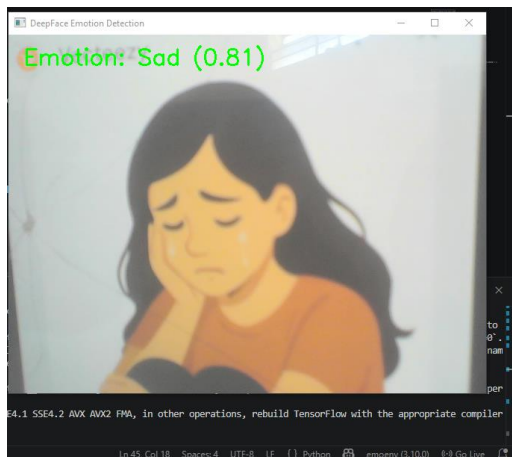
```
(emoenv) PS D:\VS Codes\Multimodal-Emotion-Recognition-System-main> python src/text/predict.py
Loading weights: 100%
Enter text: It was not at all good to see that house burning.
Predicted Emotion: Sad
Enter text: It was so irritating
Predicted Emotion: Disgust
Enter text: aww..soo cute
Predicted Emotion: Happy
Enter text: my professor didn't give us grade this sem
Predicted Emotion: Neutral
Enter text: me and my teammates finally completed the project
Predicted Emotion: Happy
Enter text: exit
(emoenv) PS D:\VS Codes\Multimodal-Emotion-Recognition-System-main>
```

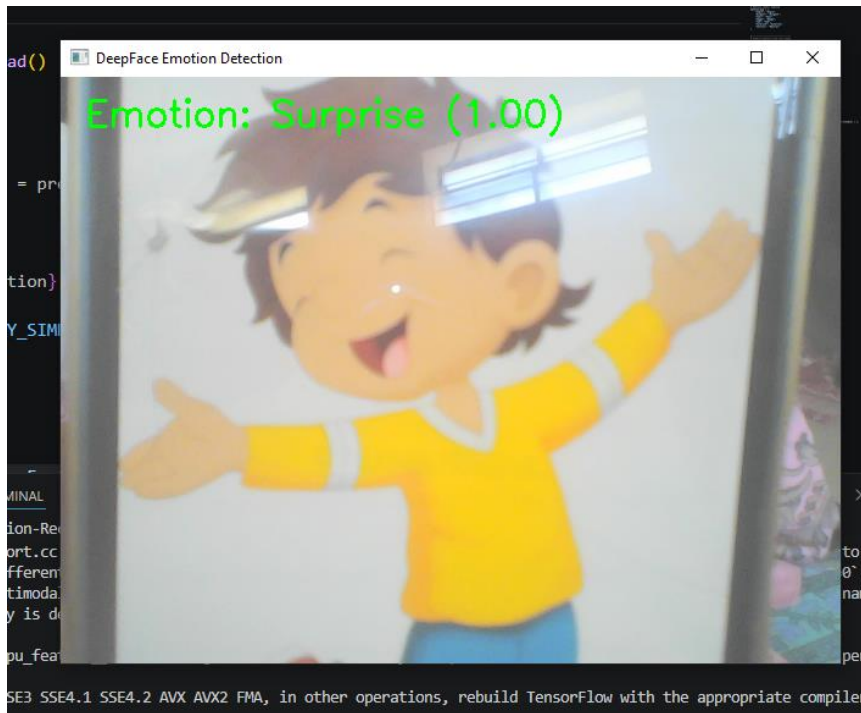
4.3 Facial Emotion Recognition Module

Example output:

Predicted Emotion:
Happy
Confidence:
0.88

Sample real-time detection screenshot:





4.5 Web Application Interface

Frontend Implementation

The frontend interface was implemented using:

- HTML
- CSS
- JavaScript

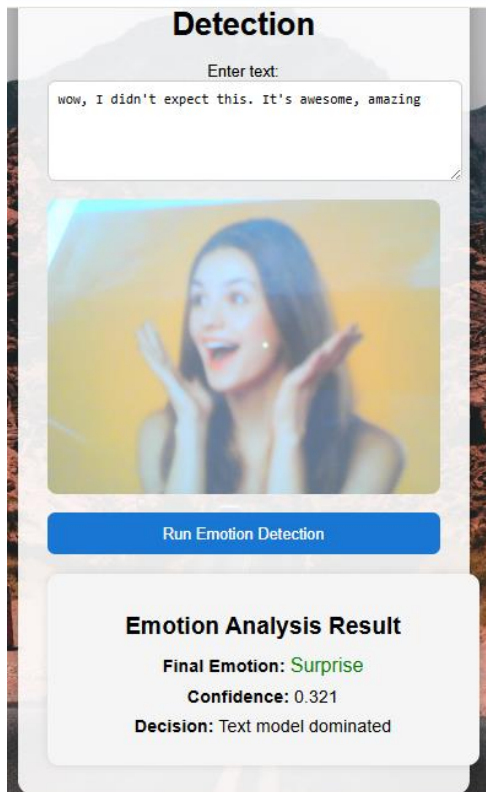
The interface provides:

- Text input area
- Webcam video display
- Emotion prediction button
- Final emotion display
- Confidence display

- Decision information display

The webcam stream is captured using the browser media devices API.

Example frontend screenshot:



Detection

Enter text:

I'm not feeling well



Run Emotion Detection

Emotion Analysis Result

Final Emotion: **Sad**

Confidence: 0.365

Decision: Text model dominated

Detection

Enter text:

oh



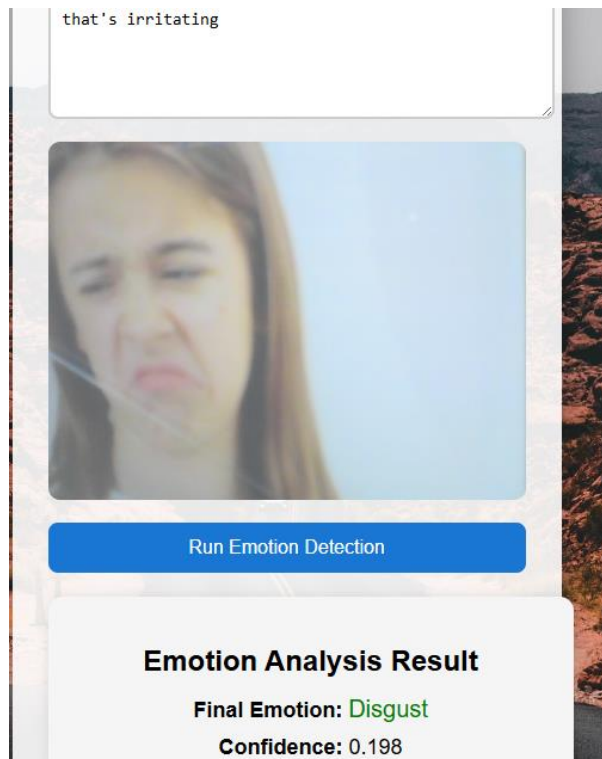
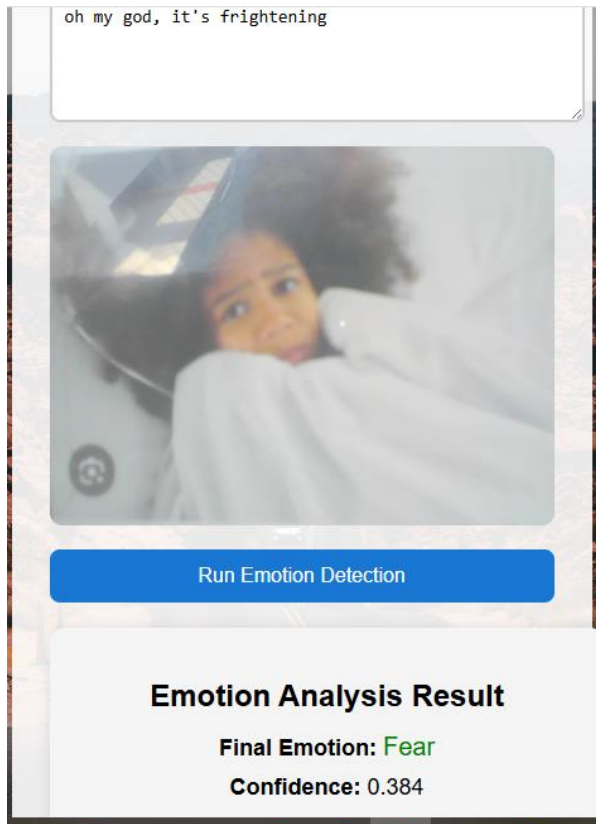
Run Emotion Detection

Emotion Analysis Result

Final Emotion: **Neutral**

Confidence: 0.35

Decision: Both models agreed



Backend Implementation

The backend server was implemented using the Flask web framework.

The Flask backend performs:

- Receiving user text
- Receiving webcam image frames
- Base64 image decoding
- Facial emotion prediction
- Text emotion prediction
- Fusion logic execution
- JSON response generation

The /predict API endpoint handles multimodal emotion prediction requests.

The backend integrates:

- RoBERTa NLP model
- DeepFace emotion model
- Fusion module

4.7 Experimental Results

Before Optimization

Terminal output screenshot:

```
(emovnv) PS D:\VS Codes\Multimodal-Emotion-Recognition-System-main> python src/optimization/measure_constraints.py
Loading weights: 100%|██████████████████████████████████████████████████████████████████████████████| 201/201 [00:00<00:00, 789.88it/s]

----- BEFORE OPTIMIZATION -----

Loading weights: 100%|██████████████████████████████████████████████████████████████████████████████| 201/201 [00:00<00:00, 672.65it/s]
Accuracy: 70.45%
Model Size: 478.92 MB
Inference Time: 716.02 ms
RAM Usage: 806.03 MB

----- AFTER OPTIMIZATION -----
```

After Optimization

Terminal output screenshot:

```
-- -- AFTER OPTIMIZATION -- --
```

```
Loading weights: 100%|███████████████████████████████████████████████████| 201/201 [00:00<00:00, 748.40it/s]
D:\VS Codes\Multimodal-Emotion-Recognition-System-main\src\optimization\measure_constraints.py:192: DeprecationWarning: torch.ao.quantizatio
n is deprecated and will be removed in 2.10.
```

```
For migrations of users:
```

1. Eager mode quantization (`torch.ao.quantization.quantize`, `torch.ao.quantization.quantize_dynamic`), please migrate to use torchao eager mod
e `quantize_API` instead
2. FX graph mode quantization (`torch.ao.quantization.quantize_fx.prepare_fx`,`torch.ao.quantization.quantize_fx.convert_fx`, please migrate to
use torchao pt2e quantization API instead (`prepare_pt2e`, `convert_pt2e`)
3. pt2e quantization has been migrated to torchao (<https://github.com/pytorch/ao/tree/main/torchao/quantization/pt2e>)
see <https://github.com/pytorch/ao/issues/2259> for more details

```
quantized_model = torch.quantization.quantize_dynamic(
D:\VS Codes\Multimodal-Emotion-Recognition-System-main\emoenv\lib\site-packages\torch\_utils.py:455: UserWarning: TypedStorage is deprecated
. It will be removed in the future and UntypedStorage will be the only storage class. This should only matter to you if you are using storag
es directly. To access UntypedStorage directly, use tensor.untyped_storage() instead of tensor.storage()
```

```
device=storage_device,
```

```
D:\VS Codes\Multimodal-Emotion-Recognition-System-main\src\optimization\measure_constraints.py:192: DeprecationWarning: torch.ao.quantizatio
n is deprecated and will be removed in 2.10.
```

```
For migrations of users:
```

1. Eager mode quantization (`torch.ao.quantization.quantize`, `torch.ao.quantization.quantize_dynamic`), please migrate to use torchao eager mod
e `quantize_API` instead
2. FX graph mode quantization (`torch.ao.quantization.quantize_fx.prepare_fx`,`torch.ao.quantization.quantize_fx.convert_fx`, please migrate to
use torchao pt2e quantization API instead (`prepare_pt2e`, `convert_pt2e`)
3. pt2e quantization has been migrated to torchao (<https://github.com/pytorch/ao/tree/main/torchao/quantization/pt2e>)
see <https://github.com/pytorch/ao/issues/2259> for more details

```
quantized_model = torch.quantization.quantize_dynamic(
Accuracy: 72.73%
```

```
Model Size: 230.94 MB
Inference Time: 21380.92 ms
RAM Usage: 618.73 MB
```

```
(emoenv) PS D:\VS Codes\Multimodal-Emotion-Recognition-System-main>
```